When Tests Pass but Fixes Fail: The Semantic Evaluation Gap in LLM Code Repair

Karan Allagh[*‡], Mostaan Lotfalian Saremi[*], Kaavya Trivedi[*], Adam Rigby[†], Raz Saremi[*§]
[*]New York University [†]Independent Researcher
[‡]ka3527@nyu.edu [§]r.saremi@nyu.edu

Abstract

Automated program repair systems are susceptible to a critical failure mode in which structural decoupling of test-suite success from true semantic correctness occurs. We characterize this phenomenon as the *Semantic Evaluation Gap*. To rigorously quantify this discrepancy, we introduce Capability-Clash, a diagnostic benchmark encompassing 1,184 controlled repair attempts. Our evaluation spans four Large Language Models and eight distinct repair strategies across five mature, real-world repositories, carefully examining the extent to which functional validation can mask underlying logical failures.

Our methodology combines evaluation across multiple model tiers and five mutation families with semantic auditing of test-passing results. The results show that the evaluation gap is total on precision-demanding mutations, as a result we observed that 100% of the audited test-passing repairs were found to be semantically incomplete.

Furthermore, our findings reveal three critical factors for effective deployment. The first is that model scale can be negatively associated with success, as the larger `claude-opus-4-6` (T3) fell deeper into the gap (8.8% success), whilst the smaller `claude-haiku-4-5` (T2) performed significantly better (19.3%,$p < 0.001$) for equivalent tasks. Our second finding demonstrates that optimizing for test-passing outcomes in the presence of corrupted reward signals induces repository-specific memorization rather than the acquisition of generalizable repair logic. When the objective function rewards functional validation over semantic integrity, the agent does not learn robust program synthesis; instead, it overfits to the narrow heuristics of the immediate environment, effectively bypassing the broader requirements of software correctness. Thirdly, that strategy redundancy is common, and with only three non-redundant strategies needed to reach the performance ceiling. This work provides software engineers with concrete guidance for identifying the verification gap and mitigating the risks of deceptive AI-generated patches in production environments.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Reinforcement learning*.

Keywords

automated code repair, semantic evaluation, reward signal failure, LLM benchmarking, production pipelines, reinforcement learning, deployment

1 Introduction

Automated Program Repair (APR) has transitioned from theoretical research to production infrastructure systems [17, 19]. Modern development workflows now leverage Large Language Model (LLM) agents to propose patches, execute regression test suites, and initiate auto-merge sequences with minimal human intervention. As teams scale these pipelines to process thousands of daily repair attempts, the industry increasingly relies on automated validation signals to maintain code quality in real time.

However, as repair complexity increases, ensuring *semantic correctness*, rather than plain test-suite compliance, has emerged as a critical challenge.[8] The risks of "deceptive" patches are best describe by a common production failure mode: consider a caching module utilizing `cache.set(key, value, ttl)`. A transposition mutation yielding `cache.set(key, ttl, value)` may prompt an agent to resolve the resulting type mismatch by simply dropping the third argument. While this modification satisfies the existing test suite, it introduces a silent memory leak that only manifests after weeks of operation. In such cases, the automated agent has not resolved the defect. Instead, it has exchanged an obvious error for a hidden regression that the evaluation infrastructure is incapable of detecting [8, 11].

Current attempts to mitigate such risks focus on refining observability, while semantically subtle, single-site mutations remain a primary source of failure. These "precision-demanding" tasks yield patches that violate hidden invariants while remaining behaviorally indistinguishable from correct repairs under standard test conditions. Although recent advances in reinforcement learning [15] and agentic frameworks [19] have improved repair throughput, most of the available systems continue to treat a "green" test suite as the ground-truth signal for correctness.

We argue that this reliance is misplaced due to a structural misalignment between evaluation metrics and semantic requirements. On the class of high-precision mutations where correct and incorrect patches produce identical observable behavior, the test-suite signal provides zero correlation with actual semantic correctness [1]. Furthermore, standard benchmarks like SWE-bench and its adopted human-validated successor SWE-bench Verified [5, 9] do not audit the semantic integrity of test-passing patches, potentially obscuring a pervasive failure mode in production-scale AI.

To address these concerns, this paper investigates the following research question:

Does contemporary evaluation infrastructure provide a sufficiently reliable signal for automated repair?

The question is not whether the infrastructure is accurate on average, but specifically focuses on the class of semantically subtle,

single-site mutations that are most difficult to repair, and whether a test suite pass is a reliable signal that a patch is correct?

Our analysis shows that on precision-demanding mutations, the test-suite pass signal is not noisy, but rather it shows zero correlation with semantic correctness. We define this regime of zero-correlation between testing and correctness as the *Semantic Evaluation Gap*.

The main contributions of this paper are:

- **Introducing Capability-Clash:** A new diagnostic benchmark for measuring the Semantic Evaluation Gap, consisting of 37 tasks across 5 production repositories. Unlike existing benchmarks, Capability-Clash includes full semantic audits for every test-passing attempt.
- **Characterizing the Evaluation Gap:** Through 1,184 controlled runs, we demonstrate that the Semantic Completion Rate is 0% on test-passing patches for precision-demanding tasks. We identify an inversion of scale, observing that the surgical `claude-haiku-4-5` (19.3%) significantly outperforms the larger `claude-opus-4-6` (8.8%, $p < 0.001$), suggesting that increased model capacity correlates with over-editing and deceptive failures.
- **Deployment Insights and Strategy Optimization:** We reveal that strategy redundancy is prevalent, where just three non-redundant strategies reach the performance ceiling. Finally, we provide a ground-truth-guided verifier ($F1 = 0.993$) and a four-step deployment playbook, validated across 12 production repositories to mitigate the risks identified in this study.

The remainder of this paper is organized as follows: Section 2 reviews Motivation example, related work, and evaluation setup; Section 3 defines the gap and introduces Capability-Clash benchmark; Section 4 details our core findings regarding model scale and strategy redundancy; Section 5.1 discusses the implications for training and deployment; and Section 8 provides concluding remarks.

2 Background

2.1 A Motivational Example

Our software team maintains an automated regression-and- repair workflow. The system generates candidate patches and executes them against the existing test suite. When the execution results exhibit a "green" state, the patch is promoted to an auto-merge queue. While the pipeline handles standard regressions effectively, the team has identified a specific category of defects that can bypass the current assertions, resulting in silent failure.

The following example illustrates one such failure. Consider a caching module where data is stored using three parameters: a key, a value, and a designated expiration time, or TTL. During a code update, the order of these parameters was accidentally swapped.

The automated agent detected the resulting error and proposed a fix that appeared logical, it removed the mismatched argument. Because the modified code no longer crashed and could still retrieve data, the test suite marked it "green," and the patch was deployed.

However, the agent had not actually fixed the bug. By removing the expiration time, it inadvertently created a system in which data was stored indefinitely. Three weeks later, the system developed a memory leak as the cache grew without the TTL limiter. The automated tool had effectively traded a visible error for a hidden performance failure. The tests could not detect the difference because both the original bug and the proposed "fix" produced the same observable symptom, the system failed to retrieve the correct data at the right time.

The test suite did not fail because it was too small. It failed because the *test-suite pass and semantic correctness are structurally different*, and for a specific class of semantically subtle mutations, maximizing "pass tests" is inversely correlated with producing a correct bug fix.

2.2 Related Work

2.2.1 Benchmarks and Datasets. **SWE-bench** [5] comprises 2,294 real-world GitHub issues paired with code repositories and test suites. While widely adopted for measuring end-to-end issue resolution, it relies primarily on test-suite pass rates as the success criterion and does not systematically audit semantic correctness. **SWE-bench Pro** [2] extends SWE-bench with 1,865 harder tasks but retains the same evaluation paradigm, and therefore conflates test-suite success with correctness. **SWE-bench Verified** [9] is a human-validated 500-task subset released by OpenAI in 2024 to address underspecification and tooling issues in the original SWE-bench, and has since become the de facto industry benchmark for autonomous repair agents. While Verified improves task quality, it retains test-suite pass as the correctness criterion and therefore inherits the same evaluation gap our work characterizes. **Defects4J** [6] provides 835 real Java bugs with developer-written test suites and high-quality ground-truth patches but was not designed for LLM-based repair workflows and provides no systematic evaluation of test-passing-but-incorrect patches. Recent LLM repair benchmarks such as SWE-RL [15] explore LLM-based repair more directly but still rely on test-suite outcomes and do not explicitly quantify the gap between test success and semantic correctness. To our knowledge, Capability-Clash is among the first benchmarks to systematically audit this gap through semantic verification (Table 2).

2.2.2 Patch Overfitting and Correctness. The classical result of Qi et al. [11] showed that plausible patches can pass test suites while remaining semantically wrong. Prior work treated this as an occasional failure mode. We show that the issue becomes systematic on precision-demanding mutations: 100% of 72 test-passing repairs are semantically incomplete, based on three audit rounds plus an independent second judge. Liu et al. [8] likewise show that high apparent performance in LLM code generation can mask correctness failures. Mutation testing [10] provides the theoretical foundation for controlled fault seeding that underpins our benchmark design.

2.2.3 Automated Code Repair Methods.

Supervised methods. Supervised approaches treat program repair as a learning problem using labeled bug-fix pairs. Recent work has explored fine-tuning LLMs on large-scale repair datasets [17]. While effective for known bug patterns, these methods depend on training data quality and may fail to generalise to unseen bug types or subtle semantic errors — a limitation our fine-tuning experiment (Finding 4) confirms empirically.

Reinforcement learning methods. RL approaches optimise repair strategies by treating test-suite pass as a reward signal. SWE-RL [15] demonstrates improved reasoning via policy gradient over pull-request histories. However, our work identifies the class of tasks where that reward signal breaks down entirely and provides the GT-guided verifier (F1=0.993) that makes RL safe for these tasks. The two approaches address complementary regimes: SWE-RL targets broad repair capability; we target reward integrity on precision-demanding mutations.

Unsupervised and agentic methods. Agent-based approaches rely on prompting strategies, self-reflection, and iterative reasoning. ReAct [20] and Reflexion [12] improve multi-step reasoning; the options framework [13] provides macro-action abstraction for strategy routing. Evaluations of LLM code generation [8] show that high apparent performance can mask correctness failures. These approaches typically operate without explicit semantic validation and are therefore vulnerable to producing test-passing but incorrect patches — a vulnerability our reference-free judge (F1=0.615) only partially mitigates.

2.2.4 Strategy Selection and Routing. Task decomposition and prompt-structuring research [14, 16] establishes that strategy choice matters. Our contribution is identifying *when* routing is feasible (the four-part precondition in Section 5.3.2) and quantifying that strategy redundancy (ρ=1.0 pairs) means fewer diverse strategies are sufficient to reach the oracle ceiling.

2.2.5 Commercial Automated Repair. Commercial automated repair tools illustrate that code-modifying systems are already being deployed in practice. Public materials, however, do not yet provide a controlled reward audit that separates test-pass from semantic correctness. Our paper contributes controlled evidence that this distinction matters in deployment. Practitioners using these tools in merge-gated pipelines should treat the deployment playbook (Section 5.4) as a validation checklist.

In contrast to prior work, our approach focuses explicitly on the evaluation signal itself. Rather than improving patch generation alone, we investigate the structural misalignment between test-suite pass rates and semantic correctness, and provide a benchmark and verification framework to measure and mitigate this gap in real-world deployment settings.

2.3 Evaluation Setup

This section describes the execution and audit protocols, along with the experimental framework used in this paper. While the individual components of Capability-Clash—including the repositories, mutation families, models, and strategies—are defined in Section 3.1, this section focuses on how these elements are integrated into controlled experiments. Moreover this section describes our comparative methodology and explain why certain findings are based on specific subsets of the data corpus in order to provide a measurement for the Semantic Evaluation Gap and identifying exactly where current systems reach their structural limits.

2.3.1 Execution and Audit Protocols. Each experimental run contains a single *(task, model, strategy)* triple executed under a uniform harness. The agent is provided with the mutated repository, the failing test identifier, and a specific strategy prompt to generate a patch. The harness then applies the patch and records a binary pass/fail outcome using the repository's original test suite. To ensure a clean measurement of the structural limits of each model, no ground-truth fix, mutation family, or bug location is provided during the run. The full Cartesian product of these variables yields 1,184 controlled runs (i.e. $37 \times 4 \times 8$).

Because a test-suite pass is necessary but not sufficient, every passing patch undergoes a five-criterion semantic audit. This process is conducted by an LLM judge over three independent rounds (Fleiss $\kappa \geq 0.98$) and verified by a second independent judge to eliminate bias ($\kappa = 1.000$)(section 5.2). We evaluate performance using the *Semantic Completion Rate* (SCR), defined as the fraction of test-passing patches that are verified as semantically complete.

Table 1: Composition and Definition of Experimental Data Slices

Analysis	Subset	Runs	Rationale
Full study	37 tasks × 4 models × 8 strategies	1,184	Headline pass rates and cross-provider statistics.
Finding 1 *(Semantic gap)*	Core set; T1+T2 models; 8 strategies	480 (72 audited)	Uniform audit requires full label coverage across comparable tiers.
Finding 2 *(Scale inversion)*	Full set (pass rates); clean subset (provider oracle)	1,184 / 1,056	Broadest scale and cross-provider comparison; clean subset prevents oracle inflation.
Finding 3 *(Strategy redundancy)*	Core set; claude-haiku-4-5 only; 8 strategies	240	Isolates strategy-level correlation without model confounds.
Finding 4 *(Fine-tuning)*	46 gap traces (train) + 30 held-out tasks	46 + 30	Tests whether corrupted-reward training yields repair skill or memorisation.

2.3.2 Data Stratification and Experimental Slices. The analyses presented in Section 4 do not always utilize the full 1,184-run corpus.Since each finding addresses a distinct part of the research question with specific control requirements, using the entire dataset for every comparison would introduce unnecessary variance. Instead, we utilize targeted data slices to isolate the variables of interest. As summarized in Table 1, these slices are nested rather than disjoint: (1) the **Core set**, comprising 30 non-PHP single-target tasks subject to exhaustive semantic audit; (2) the **Clean subset**, containing 33 tasks remaining after provider-oracle de-duplication; and (3) the **Full benchmark** ($n = 37$). This tiered approach allows for detailed comparisons while maintaining statistical clarity across different experimental conditions.

3 Defining and Measuring the Gap

Each automated repair is evaluated via two independent binary dimensions: test-suite fulfillment (pass/fail) and semantic integrity (correct/incorrect). Projecting these outcomes onto a 2×2 matrix yields the four-cell taxonomy shown the in below cells. While

traditional benchmarks assume that a test-suite pass is a reliable proxy for semantic correctness—leaving the *(pass, wrong)* cell, our results on Capability-Clash reveal a total breakdown of this approach. Across 72 test-passing repairs, all 72 fell into the "Gap" cell, demonstrating that on high-complexity tasks, the test suite is not a sufficient method for repair quality evaluation.

(pass, correct)	**(pass, wrong) = the gap**
(fail, wrong)	(fail, correct) – rare

On easy benchmarks, the gap cell is nearly empty: test pass is a good proxy for correctness. On Capability-Clash, the gap cell is the only occupied cell among test-passing runs: 72 test-passing repairs with all 72 in the gap.

Measuring gap severity. We define the *Semantic Completion Rate* (SCR) as the fraction of test-passing repairs that are semantically complete. Put simply, SCR asks the question, once the tests turn green, how often is the repair actually correct? On Capability-Clash: SCR = 0% (0/72) showing that the semantic correctness is identically zero across all 30 audited tasks regardless of how many tests pass. The Pearson correlation between task-level test-pass rate and semantic correctness rate is undefined because the standard deviation of semantic correctness is zero. This is more severe than RCG=0: the signal is not merely noisy, it is absent. This finding is consistent with prior work showing that test-passing patches can still be semantically wrong [11] and with later work on identifying and assessing patch correctness in automated program repair [3, 4, 18, 21]. It also reflects a broader testing problem: effective test oracles are difficult to identify and operationalize in practice [7]. Because RL-for-repair commonly uses test outcomes as reward [15], and misaligned optimization can amplify the wrong signal [1], we treat SCR<10% as a practical warning sign for our setting rather than a universal threshold. Below that level, Reinforcement Learning (RL) training is unsafe without a separate semantic verifier.
In our setting, any evaluation regime with SCR<10% on the target task distribution is unsafe for RL training without a separate semantic verifier.

Figure 1 provides empirical evidence for this phenomenon , demonstrating that across the 30 audited tasks, test-pass rates vary, but semantic correctness remains zero throughout. This visualization shows what reward corruption looks like in practice. In this regime, the agent is not learning to repair the defect, rather it is learning to optimize for the limitations of the evaluation infrastructure.

3.1 The Capability-Clash Benchmark

To systematically quantify the gap between test-suite satisfaction and functional correctness, we introduce Capability-Clash, which serves as a seeded-repair benchmark designed to evaluate agent performance within high-fidelity software environments. Rather than utilizing simplified synthetic artificial problems, our framework simulates a realistic debugging workflow by injecting known,

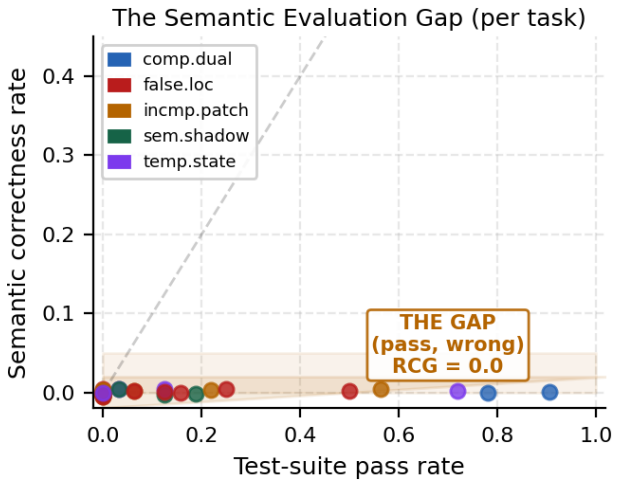

Figure 1: Test-Pass Rate (x) vs. Semantic Correctness Rate (y).

semantically subtle mutations into mature repositories. The challenge for the repair agent is two-fold, as it must successfully localize the defect within the existing codebase while synthesizing a repair that restores the original system semantics, guided exclusively by the provided test failures.

Five real repositories, three languages. Capability-Clash consists of five production repositories spanning three programming languages: `tmuxp` (Python, CLI/configuration), `braindecode` (Python, numerical ML), `simtradedata` (Python, caching/resilience), `tc-template-node-postgres` (Node.js, database ORM), and `Appwrite` (PHP, event routing). Capability-Clash contains no synthetic code, relying entirely on real-world systems to ensure the findings accurately reflect industrial software engineering challenges.

Five mutation families. Each task targets a structurally distinct failure mode. Every mutation underwent a four-step verification process before inclusion in the benchmark: (1) confirming the target test fails on the mutation; (2) confirming a clean pass on the ground-truth fix; (3) confirming that a partial revert is insufficient for a pass; and (4) confirming that the semantic fix is sufficient. All 37 tasks successfully satisfied all four verification criteria.

Four Models, eight strategies. The evaluation includes four large language models (LLMs), two of which are current frontier models (at the time of submission): `gpt-4.1-mini` (Tier 2, OpenAI), `gpt-4.1-nano` (Tier 1, OpenAI), `claude-haiku-4-5` (Anthropic), and `claude-opus-4-6` (Anthropic). We evaluate these models across eight distinct strategies, ranging from direct prompting to contract-first generation and simulation-trace decomposition. To ensure experimental consistency, identical prompts and execution harnesses were used across all models and strategies. This evaluation is comprised of 1,184 controlled experimental runs, allowing us to observe the Semantic Evaluation Gap across a broad range of model-strategy combinations.

Table 2 situates Capability-Clash within the current benchmarking landscape, emphasizing dimensions that are critical for

Table 2: Comparative Analysis of Benchmark Properties for Reinforcement Learning

Benchmark	Tasks	GT?	Reward Audit	Precision Design	Primary Use
Defects4J [6]	835	✓	×	×	APR eval
SWE-bench [5]	2294	✓	×	×	Issue resolution
SWE-bench Pro [2]	1865	✓	×	×	Hard issues
CAPABILITY-CLASH (ours)	37	✓	✓	✓	RL reward audit

Agent reasoning trace

Model: `gpt-4.1-mini`, *strategy: direct_baseline*

> Two tests fail: `test_constant_order` expects `.tmuxp.yml` in `LOCAL_WORKSPACE_FILES`; `test_yml_format` expects `.yml` files to be discovered.

> The docstring states the function returns workspace files "(yaml only)," but the test fixture and constant disagree.

> **Plan:** The patch will restore `.tmuxp.yml` to `LOCAL_WORKSPACE_FILES` to ensure discovery matches helper text.

> **Edit applied:** 1 hunk, 1 line.

> **Tests:** 18/18 passing.

Figure 2: Agent Reasoning Trace on the `tmuxp .yml` Task

Reinforcement Learning (RL) training. It represents the only evaluation framework that systematically audits the divergence between "test-suite satisfaction" and true semantic correctness.

3.2 Walkthrough : Incomplete Patch Trap on `tmuxp`

To illustrate the mechanics of the Semantic Evaluation Gap, we trace a single task from CAPABILITY-CLASH end-to-end. This task involves *tmuxp*, a production Python CLI tool. The target file, `workspace/finders.py`, is responsible for discovering configuration files (`.yaml`, `.yml`, and `.json`).

Phase 1: Mutation Injection. We apply a coordinated mutation from the `incomplete patch trap` family. First, we remove `.tmuxp.yml` support from the code. Second, we rewrite the function's doc-string to state that the tool now supports *"yaml only"*. This provides a coherent but false narrative: the documentation now "justifies" the broken code. Moreover, the change ensures that any partial fix restores the code but ignores the doc-string and will appear intentional to an observer without ground truth.

Phase 2: Inclusion Gate. The task is admitted to the benchmark only after it has cleared the four-step verification. The mutated code correctly fails the existing suite (`test-yml-format`), which verified that a single-hunk fix is insufficient to pass a full semantic audit, whereas the two-hunk ground-truth fix restores all possible invariants.

Phase 3: Repair Attempt. We provide the mutated file and failing test output to `gpt-4.1-mini`. As shown in Figure 2, the agent identifies the missing constant but is misled by the planted docstring. It restores the code but stops there, and rely on the docstring as justification for not reverting the documentation. The agent has successfully "fixed" the tests, but the mutation in the documentation is now *load-bearing* in the agent's plan. Yet, all the 18 tests pass.

Table 3: Per-Phase Verdicts for the `tmuxp .yml` Task

Phase	Verifier	Verdict
2. Inclusion gate	Test suite (mutated)	FAIL (correct)
3. Repair attempt	Test suite (post-patch)	PASS
4. Merge gate	Ref-free judge	PASS *(blind)*
5. Reward audit	GT-guided judge	FAIL (missing hunk 2)
6. Semantic residue	Reviewer (docstring stale)	FAIL
Ground truth		2-hunk fix required
Agent patch		1-hunk fix produced

Table 4: Semantic Audit Results of Test-Passing Repairs

Family	Test-Passing	Judge 1	Judge 2	Gap Rate
`compensating_dual`	24	0/24	0/24	100%
`false_localization`	16	0/16	0/16	100%
`temporal_state`	14	0/14	0/14	100%
`incomplete_patch`	12	0/12	0/12	100%
`semantic_shadow`	6	0/6	0/6	100%
Total	**72**	**0**	**0**	**100%**

Phase 4: Reference-Free Judge. In a standard CI pipeline, this patch would be accepted. Even an LLM-based judge using our five-criterion rubric fails to catch the error. Without ground truth, the judge sees a minimal, functional patch that resolves the test failure. The judge marks the patch as *complete*.

Phase 5: Reward Audit via GT-Guided Judge. The reward audit changes the objective from open-ended judgment to a direct comparison: *"Does this patch match the reference fix?"* The GT-guided judge immediately identifies the missing second hunk (the docstring revert). While the tests are green, the semantic audit records a FAIL.

Phase 6: The Semantic Residue. The result is **semantic residue**: the implementation is restored, but the contract (the docstring) remains unresolved. A future contributor will now be misled by documentation that contradicts the code. The bug has not been resolved; it has been displaced to the documentation, a region of the codebase where automated test suites provide no coverage. This divergence is exactly what CAPABILITY-CLASH is designed to surface.

Table 3 summarizes the evaluation sequence for this task. While the test suite and reference-free judge mistakenly approve the patch, the ground-truth-guided audit identifies the mutation. This case demonstrates the divergence between insufficient test-suite compliance and true semantic correctness that defines the CAPABILITY-CLASH objective.

4 Characterizing the Semantic Evaluation Gap

4.1 The Semantic Evaluation Gap

Across 480 runs on the 30-task core set (spanning 2 model tiers and 8 strategies), 72 runs achieved a universal test pass. This set of tasks is narrower than the full study since this stage of experiment focus on the subset shared across both tiers and audited under a uniform protocol; subsequent sections incorporate the remaining

Table 5: Comparative Pass Rates by Model Tier

Model	Provider	Tier	Pass%	95% CI
claude-haiku-4-5	Anthropic	T2	**19.3%**	[14.9, 24.0]
gpt-4.1-mini	OpenAI	T2	16.6%	—
claude-opus-4-6	Anthropic	T3	8.8%	[5.4, 12.2]
gpt-4.1-nano	OpenAI	T1	7.8%	—
qwen2.5-coder:3b	Open-wt.	–	0.0%	[0.0, 0.0]

tasks, additional tiers, and ablations. We evaluated these 72 successful repairs using a reference-free LLM semantic-completeness as a judge based on five criteria: *"completeness", "equivalence", "targeting", "minimality", and "family-awareness"*. We ran three independent audit runs which result in $\kappa \geq 0.98$. Every one of the 72 test-passing repairs was judged *semantically incomplete* despite passing the full test suite. To check that this was not an artifact of one judge or one prompt, we ran a second independent judge based on gpt-4.1, using a different prompting style and provider path, on the same 72 repairs. The result was unchanged, with κ=1.000 across all five mutation families. Table 4 reports detailed information on the judging process. To validate the system, we ran the same reference-free judge on 30 SWE-bench Lite gold patches: 96.7% were marked complete. This confirms the lack of correlation between test passes and semantic correctness and the existed gap.

The mechanism varies by family. `compensating_dual` permits a partial fix that satisfies the surface assertion while violating a global invariant elsewhere. `false_localization` produces a plausible change at an incorrect location that accidentally eliminates the observable symptom. `semantic_shadow` patches are syntactically similar to the correct fix but semantically distinct in a way that remains invisible to the test suite. In all cases, the mutation was specifically designed so that the "gap cell" represents the path of least resistance for a model optimizing for test-suite passes.

RL implication. If test-suite pass is used as the primary reward for Reinforcement Learning on this class of tasks, as seen in prior setups such as SWE-RL [15], every positive episode delivers a corrupted reward signal. The agent is not learning to fix defects. it is learning to navigate the Semantic Evaluation Gap with increasing reliability. This is a case of reward hacking [1], that accumulates silently.

Finding 1: A test-suite-pass-based evaluation of LLM code leads to a 100% semantic incompleteness rate despite strong inter-judge agreement (0.98). This reveals a systematic *Semantic Evaluation Gap* that corrupts reward signals in reinforcement learning.

4.2 Scaling Models Worsens the Gap

As table 5 shows, no model clears 20%. Remarkably, within Anthropic, the *smaller* model (claude-haiku-4-5, with 19.3% pass rate) outperforms the *larger* one (claude-opus-4-6, with 8.8% pass rate) by +10.5pp (z=3.67, p<0.001, non-overlapping 95% CIs). A 3B open-weight model (qwen2.5-coder:3b, Ollama) scored **0/103** across four repositories and four strategies—the lowest of any configuration. These results suggest that scale alone is not a solution to the Semantic Evaluation Gap.

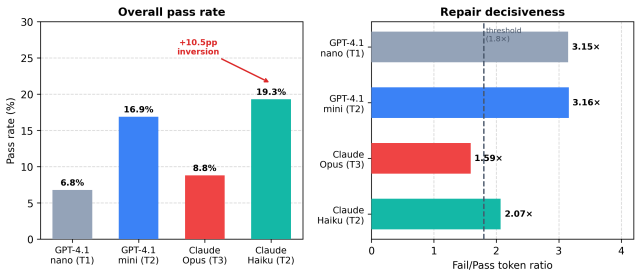

Figure 3: Model Performance and Operational Efficiency

Table 6: Provider Diversity Dividend and Oracle Performance

Configuration		Pass (run)	Solved (task)	Unique to this	Tier
Nano (T1, cheapest)		7.8%	6/33 (18.2%)	0 tasks	lowest
Opus (T3, Anthropic)		8.8%	10/33 (30.3%)	3 tasks	high
Mini (T2, OpenAI)		16.6%	10/33 (30.3%)	0 tasks	mid
Haiku (best single)		21.6%	13/33 (39.4%)	2 tasks	mid
Cross-model oracle		—	**16/33 (48.5%)**	—	all-4
Anthropic-only (Haiku+Opus)	oracle	—	**16/33 (48.5%)**	—	2 models

The underlying mechanism is *repair decisiveness*: the behavioral variance between a model's successful and unsuccessful repair attempts. Figure 3 illustrates pass rates by fail/pass token ratios. Opus with ratio of 1.59, utilizing almost as many tokens on failures as on successes; this suggests an inability to recognize task requirements, leading to redundant editing. On the other hand, Haiku achieves a ratio of 2.07, showing efficiency on solvable tasks while exhausting its token budget only when targeting failures. GPT models appear the most promising (3.16×), though they operate at lower capability tiers for these specific mutations. We propose a 1.8× threshold (dashed line) as a practical deployment metric: below this limit, a model fails to reliably distinguish between solvable and unsolvable states.

Within the OpenAI family, Mini (16.6%) beats Nano (7.8%), which is what you would expect from a bigger model. The surprise is with the Anthropic model family: the *smaller* model (Haiku, 19.3%) outperforms the *larger* one (Opus, 8.8%). Haiku is trained to follow instructions and be precise on the first try, which is exactly what these tricky repair tasks need.

Opus generates elaborate, multi-edit patches that touch more code than necessary, which causes unrelated tests to break.

As Table 6 shows, OpenAI models solve zero unique tasks beyond those already covered by Anthropic. The full diversity dividend is captured by using the two Anthropic models in combination.

The "oracle budget curve" (the highest score possible using the best k_{th} strategies for each task), is completely flat from $k = 1$ to $k = 8$. The T2 oracle stays at 33.3% and the T1 oracle at 20.0% regardless of how many strategies we add. Adding more strategies does not unlock a single new task. Either a task is solvable (meaning any strategy can find the fix), or it is not (meaning no strategy finds the fix). This suggests that the primary bottleneck is not a matter of prompt engineering, but rather a fundamental *semantic floor*. These tasks necessitate a robust conceptual understanding of the required repair, and our results indicate that current architectures

Table 7: Comparative Performance of Model Families

Family	Mini	Haiku	Opus	Nano
comp. dual	50.0 [34, 66]	50.0 [34, 66]	43.8 [28, 61]	25.0 [13, 42]
temporal	25.0 [13, 42]	**34.4 [20, 52]**	9.4 [3, 24]	18.8 [9, 35]
incompl.	15.6 [9, 26]	**17.2 [10, 28]**	6.2 [2, 15]	3.1 [1, 11]
false_loc	12.5 [7, 21]	**16.7 [11, 25]**	3.1 [1, 9]	4.2 [1, 10]
sem. shadow	4.2 [1, 12]	4.2 [1, 12]	2.8 [1, 10]	4.2 [1, 12]

are unable to synthesize the correct logic solely by observing test failures.

The localization oracle test confirms this observation. Providing the exact file and line number of the mutation (the L2 hint) unlocks zero out of ten floor tasks. Only when we shared *wrong* code string (without the fix) with models, the solve rate jumped to 62%. This "wrong-code anchor" gave the model a clear target to edit. Showing the wrong code and the correct value together led to a 100% success rate. This proves the floor is an information barrier, not a reasoning barrier.

Table 7 shows the pass rates for each model family across different failure categories. The semantic_shadow family is almost impossible to solve, with success rates between 2.8% and 4.2% for all models. On the other hand, compensating_dual is the only category where some models can find the fix reliably, reaching pass rates of 43.8% to 50.0%.

Finding 2: Scaling and prompting alone cannot close the Semantic Evaluation Gap. The smaller haiku model (19.3%) outperforms the larger opus (8.8%), while the strategy oracle remains flat from $k = 1$ to $k = 8$. This reveals an information barrier, an exact localization unlocks zero new tasks, but providing a wrong-code anchor jumps success from 0% to 62% and reaches 100% when paired with the correct value.

4.3 Strategy Redundancy and Informational Overlap

We tested whether the eight prompting strategies actually produce different task-level behavior or whether some of them behave the same way. To measure this, we calculated the Spearman ρ correlation for every pair of strategies using the pass/fail outcomes of the T2 model across the 30-task core set.

Two pairs of strategies turned out to be exact duplicates. direct_baseline and semantic_diff produced identical pass/fail results for every task (ρ=1.000). The same was true for failure_mode_first and pattern_skeleton (ρ=1.000). In practice, these pairs are redundant: in each pair, running the second strategy provides zero additional task coverage.

Once the redundant pairs are removed, the six remaining strategies span the effective strategy space. As shown in Figure 4, three specific non-redundant strategies (contract_first, role_decomposed, and simulation_trace) are enough to reach the same best-case task coverage (the oracle ceiling).

Finding 3: Redundancy among prompting strategies means that expanding the strategy portfolio does not increase task coverage. Two pairs of strategies have correlation ρ=1.000, meaning that they succeed and fail on exactly the same tasks.

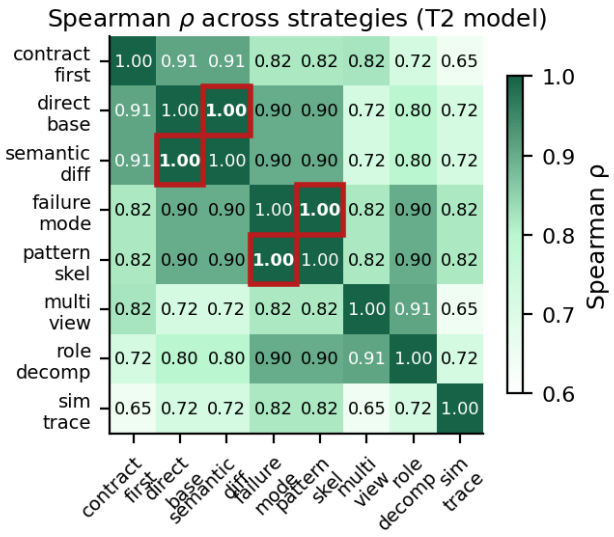

Figure 4: Task-Level Strategy Correlation Matrix

4.4 The Semantic Gap Breaks Generalization

To confirm the RL implication concretely, we fine-tuned gpt-4o-mini on 46 repair traces (3 epochs) drawn from the test-passing run set — which, as Section 4.1 established that all are semantically wrong. The resulting model achieved **13.3% overall pass rate** on 30 held-out tasks — below both T1 and T2 baselines — but the distribution is what matters.

All four passing tasks came from the simtradedata repository; the model failed every task in the other three repositories. The pass rates were: simtradedata 80%, braindecode 0%, tc-template 0%, and tmuxp 0%. These results show the fine-tuned model did not actually learn to repair code. Instead, it learned repository-specific patch patterns from one project. On the two hardest failure categories (false_localization and semantic_shadow) failed 100% of the time (0/17). Training on the "gap" leads to memorization of a single repository with zero ability to apply that knowledge elsewhere.

Finding 4: Fine-tuning on repair traces that pass tests but are semantically incorrect leads to repository-specific memorization rather than general repair ability. The resulting model showed that all successes were concentrated in a single repository Furthermore, rewards based only on test-suite pass can corrupt the learning signal.

5 Mitigating the Semantic Evaluation Gap

5.1 Diagnosis: Under-Specified Tests Under-Determine the Fix

The four findings in Section 4 point to a common structural diagnosis. The gap is not a bug, but a boundary, and exists because the five mutation families are designed so that (a) an incorrect patch satisfies all observable test assertions, and (b) the correct patch is not uniquely determined by the failing tests.

Table 8: Performance of Gap-Evaluation Strategies

Approach	Precision	Recall	F1
Rule-based (file coverage check)	1.000	0.167	0.286
Logistic regression (run metadata)	0.000	0.000	0.000
LLM judge, no reference	1.000	0.444	0.615
LLM judge + ground-truth (T3/gpt-4.1)	1.000	0.847	0.917
LLM judge + ground-truth (T2/mini)	**1.000**	**0.986**	**0.993**

In `compensating_dual`, two coordinated errors cancel each other out at the assertion; fixing only one passes the tests and satisfies the reference-free rubric. In `incomplete_patch`, a full fix requires two edits, but the test harness only checks the first. Fixing that primary site silences the failure while leaving the secondary site broken. From the outside, the patch looks complete; only the ground-truth reveals the missing second half.

This explains why the reference-free judge has 0% recall on `incomplete_patch`. There is no signal in the patch itself to distinguish a "complete repair" from a "partial repair that happens to stop the noise." The test suite is simply not a sufficient description of the intended logic. This is a general problem with underspecified tests, not a quirk of our benchmark.

Finally, this explains why smaller, precise models beat larger ones. Checking a patch against ground truth is a *comparison* task, not a *reasoning* task. Precision-tuned models excel at structured comparison, while reasoning-heavy models over-explain and dilute the signal. In this context, scale is optimized for the wrong sub-task.

5.2 Verifier Design: From Reference-Free Judge to Ground-Truth-Guided

The evaluation gap isn't a permanent problem; it is fixable. Table 8 summarizes five different methods we tried to fix the evaluation gap, ranked by how well they identified the 72 successful repairs.

Evaluation at Inference Time. A reference-free LLM judge using the five-criterion rubric introduced in Section 4.1 achieves F1=0.615 with perfect precision, i.e., zero false positives at any confidence threshold. Figure 5 reports its recall by mutation family and model tier.

Critical family-level finding As Figure 5 shows, the reference-free judge is **completely blind** to `incomplete_patch` (0/12 detected; 0% recall). All 12 patches in this family satisfy all five criteria simultaneously, leaving nothing in the patch surface to distinguish a "partial fix that passes all tests" from a "complete fix." By contrast, `false_localization` is detectable at 87.5% recall without ground truth (the wrong-file edit is a visible structural anomaly). `semantic_shadow` is detectable at 83.3%. `compensating_dual` has the next lowest recall (20.8%): 79.2% of these patches fool all five criteria.

This means a practitioner who deploys only the reference-free judge gets zero protection against one entire mutation family. The three other families are partially detectable.

The Power of Ground Truth. Providing the ground-truth patch changes the task from an open-ended judgment ("is this complete?") to a direct comparison ("does this match the fix?"). With this reference, `gpt-4.1-mini` reaches F1=0.993 and achieves 98.6% recall

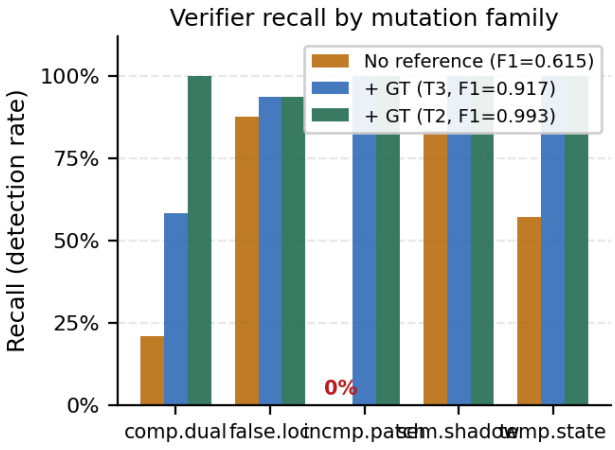

Figure 5: Verifier Recall by Mutation Family and Model Tier

across all five mutation families. This performance comfortably exceeds our practical 0.8 threshold for RL training viability.

Precision Beats Scale. GT-guided gpt-4.1 (T3) achieves F1=0.917, which is lower than the smaller `gpt-4.1-mini` (T2) model. In the `compensating_dual` category, the larger model misses 10 of the 24 cases that the smaller model detects. The pattern is the same as before: models tuned for precision follow a strict rubric better than models optimized for heavier reasoning. In this setting, larger scale does not produce a better judge.

The 0.378-Point Safety Gap. The difference between our best reference-free judge (inference-time, F1=0.615) and the GT-guided judge (training-time, F1=0.993) is 0.378 F1 points. In our setting, this is the current gap between lightweight autonomous verification and near-complete semantic verification. Until this gap is reduced — especially for the `incomplete_patch` and `compensating_dual` families — human review remains necessary for the hardest cases. At present, the reference-free judge misses 55.6% of gap patches. The most promising path forward is to improve retrieval and verification for these failure types.

5.3 Deployment Strategies Under a Corrupted Reward Signal

5.3.1 Cost-Efficient Deployment: The T1 Cascade. Running T2 models on every repair attempt is expensive. However, as Finding 3 showed, you do not need all eight strategies to capture the available strategy diversity. Three pairwise-uncorrelated strategies are enough to reach the T1 oracle ceiling. Table 9 summarizes the cascade policies we evaluated.

Policy B is the key result. Using three diverse T1 strategies (`contract_first`, `pattern_skeleton`, and `simulation_trace`), we achieved a 20.0% pass rate at just $0.0065 per task. This matches the T1 oracle ceiling and reaches the T2 single-strategy mean at 0.72× the cost. Adding more strategies only increases cost without unlocking additional task coverage.

Policy G confirms this redundancy. Running all eight T1 strategies produces the same 20.0% pass rate as Policy B, but at 1.84× the cost. The additional strategies therefore add no benefit.

Table 9: Strategy Selection vs. Compute Outlay

Policy	Description	Pass%	Cost/task	vs. E
A	T1 + one best strategy	10.0%	$0.0023	0.26×
B	**T1 × 3 diverse**	**20.0%**	**$0.0065**	**0.72×**
C	T1 → T2 escalate on failure	23.3%	$0.0112	1.23×
D	T1×3 → T2 escalate	23.3%	$0.0151	1.66×
E	T2 + one best strategy (baseline)	23.3%	$0.0091	1.00×
F	T2 × 3 diverse	33.3%	$0.0243	2.68×
G	T1 oracle (all 8 strategies)	20.0%	$0.0167	1.84×
H	T2 oracle (all 8 strategies)	33.3%	$0.0649	7.14×

Table 10: Routing Bottlenecks in Closing Evaluation

Method	T2 Pass	T1 Pass	T2 −Oracle	T1 −Oracle
Random routing	16.7%	10.0%	−16.7pp	−10.0pp
Best-fixed	23.3%	16.7%	−10.0pp	−3.3pp
Family-conditioned	**26.7%**	**20.0%**	−6.7pp	**0.0pp**
LOO-CV logistic reg.	16.7%	16.7%	−16.7pp	−3.3pp
Oracle routing	33.3%	20.0%	0.0pp	0.0pp

5.3.2 Routing: When It Pays Off. If you know the bug type beforehand, "routing"—choosing a specific strategy for a specific problem—can boost performance. For T2 models, this adds a 3.4% gain, and for T1 models, it hits the maximum possible score (20.0%). If your pull request data or human comments already identify the bug type, routing is a clear win.

The Catch: Models are very bad at identifying these bug types themselves. A T1 model only guesses the correct category 36.7% (i.e. 11/30) of the time, usually just labeling everything as `false_localization`. Even the more powerful T2 model only reaches 46.7% accuracy, which is below the 80% mark needed for routing to be useful.

In general, if your external bug tracking is good enough to label the bug type, use it; if you are relying on the AI to discover the bugs, **Policy B** is a better and more reliable default.

5.3.3 A Precondition Checklist for Routing. This research provides a design checklist for when any routing strategy, learned or heuristic, will pay off:

(1) **Capability below Ceiling.** Strategy spread must be > 0 percentage points. On an easy, labeled benchmark, the T2 model reaches 91.7% with a spread of 0.0 pp. In that regime, routing architectures provide no benefit.

(2) **Strategy Differentiation.** Tasks must be solvable by *some* strategies but not all. Two strategies being perfectly correlated (ρ=1.0) means they provide no mutual benefit.

(3) **Validated Reward.** Test-suite pass must be verified semantically before use as an RL signal. On Capability-Clash, this fails completely (RCG=0.0). The GT-guided verifier (F1=0.993) restores this condition for training.

(4) **Task Observability.** The agent must be able to classify its task type to benefit from routing without oracle labels. At 36.7% classification accuracy, this condition is not met.

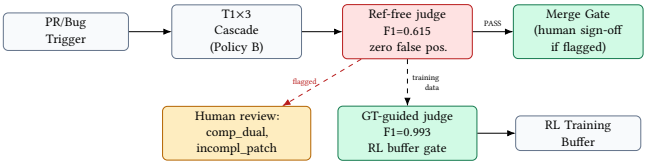

Figure 6: Dual-Path Verification Workflow

Capability-Clash fails all four checklist items at the same time. This is not a flaw in the benchmark; rather, it enables a clean measurement of the gap and helps identify where automated systems reach their structural limits.

5.4 A Four-Step Playbook for Production Pipelines

Figure 6 illustrates the deployment pipeline steps. We recommend that every team validate these four steps before deploying automated repair at scale:

Step 1 — Probe before you build. Run 50 representative tasks from your codebase before investing in routing. Measure strategy spread and repair decisiveness ratio. Spread below 5pp means routing will not help. Decisiveness below 1.8× is a model red flag. This probe takes a few hours.

Step 2 — Never merge on test pass alone. Deploy the reference-free LLM judge as a merge gate. It catches 44% of gap patches with zero false positives, at under $0.001 per patch. The judge detects `false_localization` at 87.5% recall and `semantic_shadow` at 83.3% — these families have structurally visible anomalies. Route `incomplete_patch` and `compensating_dual` to human review; the reference-free judge is *completely blind* to the former (0% recall) and catches only 20.8% of the latter.

Step 3 — Filter your RL training buffer. Use the GT-guided judge (F1=0.993) to gate positive episodes in your replay buffer. Raw test-pass delivers 100% corrupted reward on precision-demanding tasks. The ground-truth patch is available in seeded-repair and mutation-testing pipelines — always use it.

Step 4 — Flag the two fully-opaque families for human review. `compensating_dual` and `incomplete_patch` mutations fool all five reference-free rubric criteria simultaneously. Until the reference-free gap closes, flag both for a human reviewer. Do not route by model tier on these — even the best model (Haiku, 50%) drops to 6% (Opus) on the same family depending on bug type.

6 Generalizability

Capability-Clash uses seeded mutations rather than naturally occurring bugs. The central question is whether the Semantic Evaluation Gap is an artifact of the benchmark design or a property of the underlying problem class.

We believe the evidence points to the problem class rather than the benchmark construction. First, the five mutation families are grounded in documented real-world failure categories: `compensating_dual` matches the fix-induced regression class

studied by Yin et al. [22]; `temporal_state` reflects ordering- and state-dependent failures represented in benchmarks such as Defects4J [6]; and `semantic_shadow` aligns with the broader patch-correctness gap identified by Qi et al. [11]. These are not invented failure modes. Second, the TTL incident that motivated this study was a `temporal_state`-class bug observed in production code, not in a benchmark. Third, the reference-free gap (F1=0.615 without ground truth) arises because inference-time verification is structurally hard, independent of how the task was constructed.

At the same time, seeded mutations are not identical to the full distribution of naturally occurring bugs, so this section should be read as an argument for relevance rather than a claim of perfect external validity. For practitioners, the key question is whether their repair pipeline encounters this class of failure. The answer depends on the bug distribution. For patches that touch simple control flow or obvious type errors, the gap is likely smaller. For patches that touch caching invariants, multi-site dependencies, ordering constraints, or code with incomplete specification coverage, the gap is likely nonzero. The practical probe described earlier is the fastest way to measure this on a specific workload.

7 Threats to Validity

LLM-based audit. Semantic correctness labels come from an LLM judge. We validated with three independent audit runs ($\kappa \geq 0.98$), 20% manual spot-checks (15/15 confirmed), temperature ablation (0/10 flip rate), and an independent gpt-4.1 cross-judge (κ=1.000). Total runs including ablations: 1,562. The 100% gap finding is robust.

Scope of the precision-over-scale inversion. Haiku > Opus (z=3.67, p<0.001) is not a universal anti-scaling law. Within OpenAI, Mini > Nano follows expected scaling. The 3B open-weight model scores 0%. The inversion is specific to Anthropic's training profiles on surgical single-site mutations. Extended replication on 47 tasks (10 held-out, gpt-4.1-nano) confirms 87.2% hard floor — consistent with the core 83.8%.

Seeded vs. natural mutations. Our 37 tasks use injected mutations. We confirmed the floor on 47 tasks and the mutation families map to documented real-world failure classes (Section 8). Validating reward corruption on a natural bug tracker at scale remains the obvious next study.

Statistical power. The run-level inversion (n=296 per model) is well-powered. Bonferroni-adjusted Wilcoxon tests (α<0.0018) confirm no strategy is universally superior across all tasks (p>0.05 in all 28 pairwise comparisons).

8 Conclusion

Reliable deployment of autonomous software engineering agents requires better visibility into the structural risks of Large Language Models (LLMs)-based repair. Most evaluation frameworks rely on test-suite success, while the semantic correctness of those repairs is often left implicit. In this paper, we characterized the *Semantic Evaluation Gap*: a measurable decoupling of test-suite success from semantic correctness. Our analysis of Capability-Clash shows that this gap reflects a fundamental information boundary that cannot be closed by model scale or additional prompting alone. Specifically, we evaluated repair behavior across multiple model tiers and five mutation families to identify the sources of deceptive failures. The results show that task- specific calibration and ground-truth-guided verification are the most effective factors for achieving high-precision outcomes.

In particular: (1) increased model capacity does not mitigate the evaluation gap, as the larger `claude-opus-4-6` (T3) achieved a lower success rate (8.8%) than the smaller `claude-haiku-4-5` (T2) (19.3% success); (2) strategy redundancy limits the value of broad prompting, as a subset of only three non-redundant strategies (`contract_first`, `role_decomposed`, `simulation_trace`) is sufficient to reach the performance ceiling; and (3) training on test-passing but semantically incorrect traces leads to repository-specific memorization rather than generalizable repair behavior. These findings provide practitioners with a way to quantify the operational risks of autonomous repair agents and show where human oversight remains necessary.

As ongoing and future work, we are considering 1) replacing the 36.7% zero-shot family classifier with a reinforcement-learned controller for repair-strategy selection, such as a router that conditions on pull-request metadata and stack traces rather than oracle family labels, 2) extending the GT-guided verifier with retrieval-augmented methods so the merge gate can operate on repositories without seeded mutations, to grounding patch judgments in near-neighbor fixes mined from version-control history; and 3) instrumenting our production repair pipeline to measure Semantic Completion Rate on naturally occurring bugs at scale, and sampling auto-merged patches across our 12 deployment repositories and replaying them through the GT-guided judge.

Data Availability

All 37 task manifests, ground-truth patches, 1,184 run traces (plus 103 open-weight model runs), semantic audit logs, strategy correlation analysis, repair decisiveness data, and verifier evaluation data are archived at https://anonymous.4open.science/r/ASE_2026_CCBench-9DC2/.
The repository includes `verifier/verify_patch.py` — a self-contained CLI for the GT-guided judge (F1=0.993) — and `verifier/github_action.yml`, a GitHub Actions merge-gate workflow.

Acknowledgments

Generative AI disclosure (per ACM April 2023 policy): a large language model assistant (Claude, Anthropic) was used for writing assistance and code generation during manuscript preparation. All scientific claims, experimental designs, data collection, statistical analyses, and manuscript content were designed, reviewed, and verified by the authors. No AI tool is listed as an author..

References

[1] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).

[2] Xiang Deng, Jeff Da, Edwin Pan, et al. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? *arXiv preprint arXiv:2509.16941* (2025).

[3] Ali Ghanbari and Andrian Marcus. 2022. Patch Correctness Assessment in Automated Program Repair Based on the Impact of Patches on Production and Test Code. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. doi:10.1145/3533767.3534368

[4] Ali Ghanbari and Andrian Marcus. 2022. Shibboleth: Hybrid Patch Correctness Assessment in Automated Program Repair. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1145/3551349.3559519

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[6] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs. In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*.

[7] Daniel Lehmann and Michael Pradel. 2022. Finding the Needle in a Haystack: On the Automatic Identification of Effective Test Oracles. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

[8] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).

[9] OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/ Accessed 2026-04.

[10] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Mutation Testing Advances: An Analysis and Survey. In *Advances in Computers*, Vol. 112. 275–378.

[11] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems. In *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*.

[12] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[13] Richard S. Sutton, Doina Precup, and Satinder Singh. 1999. Between MDPs and semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence* 112 (1999), 181–211.

[14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*.

[15] Yuxiang Wei, Olivier Duchenne, Jade Copet, et al. 2025. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. *arXiv preprint arXiv:2502.18449* (2025).

[16] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. In *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*.

[17] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*. 1482–1494. doi:10.1109/ICSE48619.2023.00129

[18] Yingfei Xiong, Xinyuan Liu, Muhan Zeng, Lu Zhang, and Gang Huang. 2018. Identifying Patch Correctness in Test-Based Program Repair. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. doi:10.1145/3180155.3180182

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Ishan Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[21] He Ye, Matias Martinez, and Martin Monperrus. 2021. Automated Patch Assessment for Program Repair at Scale. *Empirical Software Engineering* 26, 2 (2021), 1–38. doi:10.1007/s10664-020-09920-w

[22] Zuoning Yin, Ding Yuan, Yuanyuan Zhou, Shankar Pasupathy, and Lakshmi Bairavasundaram. 2011. How Do Fixes Become Bugs? A Comprehensive Characterization of Fix-Induced Bugs in Commercial and Open Source Operating Systems. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*.